Group members: Ashwin Varkey 23115420

Chukwudi Williams 23061170

Manik Malik 23169717

Assignment 5

Q1.

```
int N = 8192;
double mat[N][N],s[N][N];
// ...
srand(1);  // random seed
for(i=0; i<N ; ++i) {
   val = (double)rand(); // random number
   for(j=0; j<N; ++j) {
      mat[j][i] = s[j][i]/val;
   }
 }
```

The issues with the code are as follows:-

- Power of two in dimensions of array: The major dimension of the arrays mat and s is N, which is 8192. Cache traffic/thrashing: In every j iteration, a full cache line is loaded from main memory and only one entry is used.
- Caches are typically organized into fixed-size blocks, and accessing data that spans multiple cache blocks can lead to cache inefficiency. When the size of the data is a power of 2, it may not align optimally with the cache block size, resulting in additional cache misses and potentially slower performance.
- Strided access due to the loop variables being interchanged (Column major order): The loops iterate over i in the outer loop and j in the inner loop. Since the arrays mat and s are accessed column-wise, the code exhibits strided access patterns. This can lead to inefficient memory access due to cache misses and potentially hinder vectorization.
- Divide operation inside the innermost loop leads to issues in pipelining.

Bad pipelining: The code has a dependency between iterations of the outer loop. The assignment mat[j][i] = s[j][i]/val depends on the previous iteration's value of mat[j][i]. This dependency prevents efficient pipelining of instructions, which can negatively impact performance.

Bad memory utilization: The code accesses the arrays mat and s in a column-major order, which can result in inefficient memory access patterns. Modern processors typically have better performance with row-major access patterns. In this case, accessing elements of mat and s in a row-major order would likely result in better utilization of the memory bandwidth.

Non-vectorizable of loops: The code performs scalar operations on individual elements of the arrays mat, s, and val. Modern processors often have vector instructions that can perform operations on multiple elements simultaneously, providing better performance. However, the given code does not take advantage of such vectorization opportunities.

Optimizations

1. Move the division operation outside the inner loop: Currently, the division operation s[j][i]/val is performed for each element in the inner loop. Since val remains constant within the outer loop, we can calculate its reciprocal (1/val) once before entering the outer loop. Then, inside the inner loop, we can multiply s[j][i] with the precomputed reciprocal value. This will reduce the number of division operations, which can be costly.

```
srand(1);  // random seed
for(i=0; i<N ; ++i) {
  val = 1./ (double)rand(); // random number
  for(j=0; j<N; ++j) {
    mat[j][i] = s[j][i]*val;
  }
}
```

2. Change the loop order and initialize random values in array outside loop: Reverse the loop order so that the outer loop iterates over j and the inner loop iterates over i. This will result in accessing the arrays mat and s in a row-major order, which can improve memory access efficiency. Generate random numbers beforehand: Instead of generating a random number for each iteration of the outer loop, we can generate all the random numbers we need before entering the loop. This will avoid the overhead of generating random numbers repeatedly.

```
// Generate random numbers beforehand
double random_vals[N];
for (int i = 0; i < N; ++i) {
   random_vals[i] = 1.0 / ((double)rand() + 1.0);
}
// Interchanged loops and optimized division
for (int j = 0; j < N; ++j) {
   for (int i = 0; i < N; ++i) {
      mat[j][i] = s[j][i] * random_vals[i];
```

The compiler will do automatically the loop interchange

Q2.

NT Store:

Register Content is stored into memory without prior write allocate
Automatic Cache Line evict on hit.

Advantages:

- Store streams don't cause cache line to get populated so more cache available for other tasks
- NT stores are faster for streaming data as they bypass the cache hierarchy and write directly to main memory.

Disadvantages:

- As stored data is not in cache so direct data reuse from cache is not possible
- It can only be beneficial if the data is not expected to be accessed soon hence has a limited applicability
- Enforces multi version loop code when array size is not constant

CL Zero:

Advantages:

- If CL is already present in cache there are not disadvantages
- Subsequent code can use stored data from cache.

Disadvantages:

- Large arrays may cause cache pollution making less space available for other data.
- Correct "boundary handling" required (CL sharing between different arrays must be avoided)

```
for(int i=0; i<N; ++i)
  a[i] = s * b[i];
```

Bc = 3 Words/Flops

NT store and CL Zero reduce Bc to 2 Words/Flops

For outer product,NT store / CL zero will not help with large value matrix multiplications.

Q3.

3. a) Without prefetching, we have 2 FLOPS in the given loop: 2 CLS to be loaded

2 CLS will require 200 cycles

Latency TL = 200*0.33 = 66 ns

Memory bus bandwidth = 24 GBytes/s

Time taken for data transfer = (2 CLs*64 bytes) / 24 GBytes/s = 5.34 ns

Total time = 66 ns +5.34 ns = 71.34 ns

For 16 Flops since 8 double precision numbers fit into one cache line and we have MULT AND ADD

Thus, the expected performance = 16 Flops / 71.34 ns = **224.278 MFlops/sec**

3.b)

$$P=T/(V/b)=1+T_L/(V/b)$$

V=Cache line length

Latency TL = 100*0.33 = 33 ns

Memory bus bandwidth, b = 24 GBytes/s

$P = 1 + \frac{33\ \text{ns}}{\frac{64\ \text{Bytes}}{24\ \text{GBytes/s}}}$ = **13.35 ~ 14**

3.c) Cache line is twice as long means 128 Bytes

The required number of outstanding prefetch with latency, $P = 1 + \frac{33\ \text{ns}}{\frac{128\ \text{Bytes}}{24\ \text{GBytes/s}}}$ = **7.179 ~ 8**

3. d) The time to fetch 2 CLs will be only required as memory access is the only bottleneck which is 5.34 ns as calculated in part(a)

For 16 FLOPs in 2 CLs:

Expected performance = 16 Flops / 5.34 ns = **2996.25 MFlops/sec**